Composition d'informatique n°2

Durée : 4h

Calculatrice non autorisée

Le sujet est très largement adapté d'un sujet Mines-Ponts Option Informatique MP

⋆ ⋆ ⋆

Préliminaires

L'épreuve est formée d'un problème unique, constitué de 32 questions, et porte sur l'apprentissage automatique (ou *machine learning*) d'un langage régulier que l'on souhaite déterminer à partir de requêtes d'appartenance et de requêtes d'équivalence.

Les applications de l'apprentissage automatique d'un langage sont étendues. En analysant le modèle appris, il est possible de détecter les divergences entre une spécification et sa mise en œuvre ou entre différentes mises en œuvre tant pour des systèmes matériels que logiciels : par exemple, protocoles de cartes à puce, réseaux de zombies sur internet (ou *botnets*), logiciels patrimoniaux (ou *legacy software*) pour ne citer que quelques illustrations.

Le problème est divisé en cinq sections reliées entre elles. Une question peut être traitée à condition d'avoir lu les définitions introduites jusque là. Dans la première section (page 2), nous établissons quelques prolégomènes. Dans la deuxième section (page 3), nous présentons une implémentation possible d'une structure de dictionnaire, les arbres bouc-émissaire. Dans la troisième section (page 7), nous nous intéressons à une relation d'équivalence induite par le langage à apprendre. Dans la quatrième section (page 8), nous étudions un certain type d'arbre de décision. Dans la cinquième section (page 9), nous construisons un automate reconnaissant le langage à apprendre.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désigne la même entité mais du point de vue mathématique pour la police en italique (par exemple n, T, δ) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`, `t`, `delta`).

Des rappels des extraits du manuel de documentation de OCaml portant sur les modules `List` et `Array` sont reproduits en annexe (page 12).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Il faudra coder des fonctions à l'aide du langage de programmation OCaml exclusivement, en reprenant l'entête de fonctions fourni par le sujet, sans s'obliger à recopier la déclaration des types. Il est permis d'utiliser la totalité du langage OCaml sauf indication contraire. Il est recommandé de s'en tenir aux fonctions les plus courantes afin de rester compréhensible. Quand l'énoncé demande de coder une fonction, sauf demande explicite, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : il est attendu que l'on choisisse des noms de variables intelligibles ou encore que l'on structure de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle.

1 Alphabets, mots et automates

Dans l'ensemble du sujet, nous fixons un *alphabet* binaire $\Sigma = \{a, b\}$ muni de l'ordre alphabétique ; la notation ε désigne le *mot vide* ; la notation Σ^* désigne l'*ensemble des mots* sur l'alphabet Σ muni de l'ordre lexicographique induit. La *concaténation* de deux mots $u \in \Sigma^*$ et $v \in \Sigma^*$ est notée uv.

Remarque OCaml : Nous adoptons les types suivants :

```
type letter = A | B
type word = letter list
```

Question 1 Écrire une fonction OCaml `cmp_letter (x1:letter) (x2:letter) : int` dont la valeur de retour est nulle si les lettres x_1 et x_2 sont égales, est strictement négative si la lettre x_1 précède strictement la lettre x_2 dans l'ordre alphabétique et est strictement positive sinon.

Question 2 Écrire une fonction OCaml `cmp_word (w1:word) (w2:word) : int` dont la valeur de retour est nulle si les mots w_1 et w_2 sont égaux, est strictement négative si le mot w_1 précède strictement le mot w_2 dans l'ordre lexicographique et est strictement positive sinon. On s'interdira l'usage de la fonction `compare` du module `List` et toute autre approche similaire.

Question 3 Décrire brièvement une structure de données mutable qui implémente un dictionnaire dont les clés sont des mots de Σ^* et dont les valeurs sont d'un type quelconque, permettant des opérations de recherche, d'insertion et de suppression en temps constant en moyenne. Quel est le nom du module OCaml permettant d'utiliser cette structure de données ?

Remarque OCaml : Dans la suite du sujet, nous nous munissons d'un module OCaml `WordMap` et d'un type `'a wordmap` qui implémentent des dictionnaires de clés de type `word` et de valeurs d'un type quelconque `'a`. Nous disposons de la constante et des deux fonctions suivantes :

- `WordMap.empty`, de type `'a wordmap`, qui désigne le dictionnaire vide.
- `WordMap.add`, de type `word -> 'a -> 'a wordmap -> 'a wordmap`, telle que la valeur de retour de `WordMap.add w v d` est un dictionnaire contenant les associations du dictionnaire d ainsi qu'une association supplémentaire entre la clé w et la valeur v.
- `WordMap.find`, de type `word -> 'a wordmap -> 'a` telle que la valeur de retour de `WordMap.find w d` est la valeur associée à la clé w dans le dictionnaire d.

Une implémentation possible du module `WordMap` est étudiée dans la partie 2.

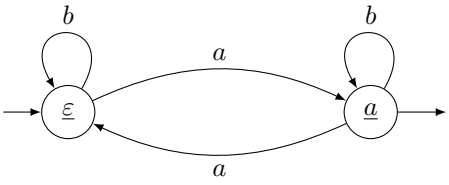
Question 4 À titre d'exemple, déclarer deux valeurs OCaml `delta_a` et `delta_b` de type `word wordmap`, et une valeur OCaml `un_a`, de type `bool wordmap`, égales aux dictionnaires respectifs :

$$\delta_a : \left\{ \begin{array}{ccc} \Sigma^* & \to & \Sigma^* \\ \varepsilon & \mapsto & a \\ a & \mapsto & \varepsilon \end{array} \right. \quad ; \quad \delta_b : \left\{ \begin{array}{ccc} \Sigma^* & \to & \Sigma^* \\ \varepsilon & \mapsto & \varepsilon \\ a & \mapsto & a \end{array} \right. \quad \text{et} \quad \mathbb{1}_{\{a\}} : \left\{ \begin{array}{ccc} \Sigma^* & \to & \{\mathsf{faux}, \mathsf{vrai}\} \\ \varepsilon & \mapsto & \mathsf{faux} \\ a & \mapsto & \mathsf{vrai} \end{array} \right. .$$

Définition : Le terme *automate* s'entend systématiquement dans le sens d'un automate fini déterministe complet, c'est-à-dire un quadruplet $(Q, q_0, \delta, \mathbb{1}_F)$ où :

- l'ensemble Q est un ensemble fini d'états,
- l'état q_0 est un élément particulier de Q appelé l'état initial,
- l'application δ est une application $Q \times \Sigma \to Q$ appelée fonction de transition,
- et l'application $\mathbb{1}_F : Q \to \{\mathsf{faux}, \mathsf{vrai}\}$ est la fonction indicatrice d'une partie F de Q appelée ensemble des états finals.

Question 5 À titre d'exemple, décrire en langue française le langage reconnu par l'automate figuré ci-dessous. On ne demande pas de justification.



Remarque OCaml : Dans ce sujet, l'ensemble des états des automates est systématiquement choisi parmi les mots de Σ^*. Typographiquement, nous convenons de souligner les mots servant d'état dans ce sujet et adoptons les types OCaml suivants

```
type state = word
type automaton = {
    initial : state;
    transitions : state -> letter -> state;
    finals : state -> bool
}
```

Question 6 À titre d'exemple, définir une valeur `automaton_example`, de type `automaton`, égale à l'automate figuré à la question 5.
On copiera et on complètera le code suivant :

```
let automaton_example = {
    initial = .... ;
    transitions = ( fun q x -> WordMap.find q (......)
                  );
    finals = .....
}
```

Définition : Pour tout automate $A = (Q, q_0, \delta, \mathbb{1}_F)$, pour tout état q de Q et pour tout mot w de Σ^*, nous notons $\delta^*(q, w)$ l'état d'arrivée dans l'automate A au terme du parcours démarrant en l'état q et suivant les transitions de A étiquetées par les lettres successives du mot w. Ainsi, la fonction δ^* est une application $Q \times \Sigma^* \to Q$ qui étend l'application δ.

Question 7 Écrire une fonction OCaml `delta_star (a:automaton) (q:state) (w:word) : state` dont la valeur de retour est $\delta^*(q, w)$.

Question 8 Écrire une fonction OCaml `accepte (a:automaton) (w:word) : bool` dont la valeur de retour est le booléen vrai si le mot w est reconnu par l'automate A et le booléen faux sinon. Il est demandé d'utiliser la fonction de la question précédente.

2 Arbres bouc-émissaire

La structure d'arbre bouc-émissaire permet d'implémenter des dictionnaires via des arbres binaires de recherche équilibrés. Cette structure a l'avantage, par rapport à d'autres arbres binaires de recherche équilibrés comme les arbres rouge-noir ou les arbres AVL, de ne pas nécessiter de données supplémentaires à stocker dans chaque nœud.

Afin de simplifier l'étude de la structure, on considère des arbres contenant uniquement des clés entières plutôt que des couples (clé, valeur).

On se donne un type d'arbre binaire de la forme :

```
type bst = E | N of bst * int * bst
```

où `E` représente l'arbre vide, et `N (l, x, r)` représente un arbre binaire ayant pour racine `x` et pour sous-arbres gauche et droite les arbres `l` et `r`.

On suppose, de plus, que tout arbre de type `bst` vérifie la condition d'être un arbre binaire de recherche au sens strict des deux côtés, c'est-à-dire que les éléments de `l` sont strictement inférieurs à `x` qui est strictement inférieur aux éléments de `r` (cette propriété étant également vérifiée pour tous les nœuds internes). En particulier, on remarque qu'un arbre binaire de recherche ne doit pas contenir d'éléments en doublon.

Un arbre bouc-émissaire est alors un couple formé d'un arbre binaire de recherche et d'un entier représentant sa taille (pour éviter de la recalculer à chaque opération).

```
type scapegoat = bst * int
```

Ainsi, si `(t, n)` est de type `scapegoat` représentant un arbre bouc-émissaire T, on a la garantie que $n = |T|$.

Question 9 Écrire une fonction `search (x:int) ((t, n):scapegoat) : bool` qui prend en argument une clé et un arbre bouc-émissaire et renvoie un booléen qui vaut `true` si et seulement si la clé apparaît dans l'arbre.

Dans la suite, sauf précision contraire, on identifie le nœud d'un arbre au sous-arbre enraciné en ce nœud.

Définition : Si $T = N(\ell, x, r)$ est un arbre binaire non vide et $\alpha \in [1/2, 1]$, on dit que T est :
- *localement α-équilibré en taille* si $|\ell| \leqslant \alpha|T|$ et $|r| \leqslant \alpha|T|$;
- *localement α-équilibré en hauteur* si $h(T) \leqslant \log_{1/\alpha}|T|$ (avec la convention que $\log_1(x) = +\infty$).

On étend ces définitions en disant que l'arbre T est globalement α-équilibré en taille (resp. hauteur) si tous les nœuds de T sont localement α-équilibrés en taille (resp. hauteur).

On remarque que tout arbre est 1-équilibré en taille et en hauteur, et qu'un arbre doit avoir tous ses niveaux de profondeur remplis, sauf éventuellement le dernier, pour être $\frac{1}{2}$-équilibré en taille.

Question 10 Montrer qu'un arbre globalement α-équilibré en taille est globalement α-équilibré en hauteur. Par un contre-exemple à moins de 8 nœuds, montrer que la réciproque est fausse dans le cas général.

Pour toute la suite de cette partie, on fixe $\alpha = \frac{2}{3}$. On implémente les arbres bouc-émissaires en garantissant qu'ils restent localement α-équilibrés en hauteur. L'idée principale est de faire les insertions comme on le ferait normalement dans un arbre binaire de recherche, et d'effectuer le rééquilibrage en taille d'un sous-arbre si l'arbre complet perd la propriété d'être localement α-équilibré en hauteur.

Question 11 Écrire une fonction `infix ((t, n):scapegoat) : int array` qui prend en argument un arbre bouc-émissaire et renvoie un tableau de taille n contenant ses éléments triés par ordre croissant. On garantira une complexité linéaire en n.

Question 12 Écrire une fonction `balance_array (tab:int array) : bst` qui prend en argument un tableau d'entiers supposé trié et renvoie un arbre binaire de recherche globalement $\frac{1}{2}$-équilibré en **taille** contenant les mêmes éléments. On garantira une complexité linéaire en la taille du tableau.

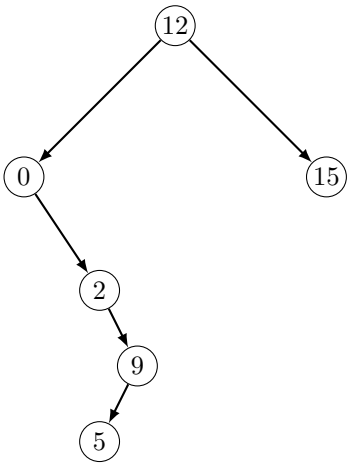
Indication : on pourra écrire une fonction auxiliaire qui prend en argument deux indices et construit un arbre équilibré contenant les éléments du tableau entre ces deux indices.

Supposons que T est un arbre obtenu après l'insertion d'un élément x dans un arbre bouc-émissaire au niveau d'une feuille, comme on le fait sans équilibrage particulier dans un arbre binaire de recherche.

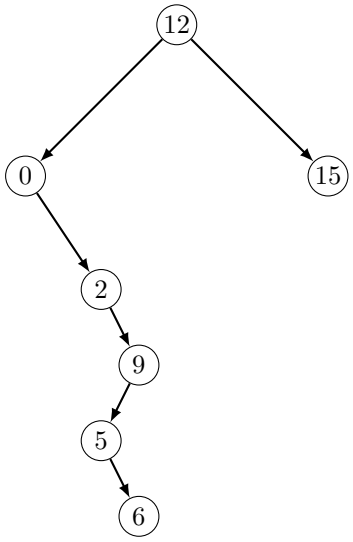
Si T n'est pas localement α-équilibré en hauteur, alors on cherche parmi les ancêtres de x un **bouc-émissaire** s, et on remplace $T(s)$ (le sous-arbre de T enraciné en s) par un arbre binaire de recherche globalement $\frac{1}{2}$-équilibré en taille contenant les mêmes éléments que $T(s)$.

L'idée est de choisir comme nœud s l'ancêtre de x le plus profond parmi ceux qui ne sont pas localement α-équilibrés en taille. On peut montrer qu'un tel nœud existe bien en accord avec la question 10.

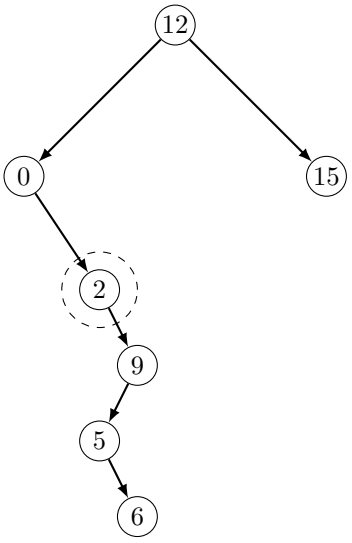
La figure 1 montre un exemple d'insertion dans un arbre bouc-émissaire.



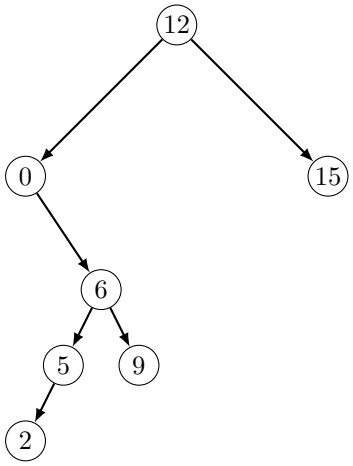
1. On veut insérer le nœud 6 dans l'arbre ci-dessus.



2. On fait une insertion classique aux feuilles.



3. L'arbre obtenu T n'est plus globalement $\frac{2}{3}$-équilibré en hauteur, car $\left(\frac{1}{\alpha}\right)^{h(T)} = 1,5^5 \simeq 7,6 > 7 = |T|$. On cherche le bouc-émissaire, qui est le nœud étiqueté par 2, car son enfant droit est de taille $3 > \frac{2}{3} \times 4$.



4. On rééquilibre l'arbre enraciné en le bouc-émissaire.

FIGURE 1 – Exemple d'insertion dans un arbre-bouc émissaire

Pour implémenter la fonction d'insertion, on se donne quelques fonctions utilitaires :

```
let threshold n =
    int_of_float (log (float_of_int n) /. log 1.5)

let unbalanced size_l size_r =
    let size_t = size_l + size_r + 1 in
    3 * size_l > 2 * size_t || 3 * size_r > 2 * size_t

let rec size = function
    | E -> 0
    | N (l, _, r) -> 1 + size l + size r
```

Un appel `threshold n` calcule le seuil $\left\lfloor \log_{1/\alpha} n \right\rfloor$. Un appel `unbalanced size_l size_r` détermine si un arbre ayant des sous-arbres gauche et droite de tailles `size_l` et `size_r` n'est pas localement α-équilibré en taille. Un appel `size t` calcule la taille d'un arbre `t`.

Question 13 Compléter la fonction `insert_and_balance` dans le code suivant dans l'objectif de rendre correcte la fonction `insert`. La fonction `insert_and_balance (depth: int) (t: bst) : bst * bool * int` prend en argument un entier `depth` et un arbre binaire de recherche `t` et fait l'insertion de l'élément `x`, en supposant que la racine de `t` se trouve à profondeur `depth` dans l'arbre donné en argument de `insert`. Cette fonction doit renvoyer un triplet formé :

- du nouvel arbre `t'` après insertion de `x` dans `t` ;
- un booléen qui vaut `true` si et seulement si un rééquilibrage est nécessaire dans un ancêtre de `t'` ;
- un entier qui est égal à la taille de `t'` si le booléen vaut `true`, et prend une valeur arbitraire sinon.

La fonction ne devra pas recalculer plusieurs fois la taille d'un même sous-arbre, ni calculer la taille d'un sous-arbre qui n'est pas un sous-arbre de $T(s)$, où s est le nœud bouc-émissaire.

On ne demande de compléter que le cas de base (ligne 3) et le cas `x < y` (ligne 6), le cas `x > y` (ligne 8) étant symétrique. On ne demande pas de recopier le code des autres lignes.

```
1  let insert x (t, n) =
2      let rec insert_and_balance depth = function
3          | E -> (* à compléter *)
4          | N (l, y, r) ->
5              if x < y then
6                  (* à compléter *)
7              else if x > y
8                  (* ne pas compléter *)
9              else failwith "Nœud déjà présent"
10     in
11     let (t', _, _) = insert_and_balance 0 t in
12     (t', n + 1)
```

Pour la suite de cette partie, on cherche à montrer que l'opération d'insertion est efficace en complexité amortie. On introduit pour cela une grandeur qui mesure le déséquilibre de l'arbre, qui peut soit peu augmenter lors d'une opération peu coûteuse, soit beaucoup diminuer lors d'une opération coûteuse.

Définition : Si $T = N(\ell, x, r)$ est un arbre binaire non vide, on appelle *déséquilibre local de* T, noté $d(T)$, la valeur :

$$d(T) = \max(||\ell| - |r|| - 1, 0)$$

On appelle *déséquilibre global de* T, noté $D(T)$, la somme des déséquilibres locaux des nœuds de T.

Par convention, les déséquilibres local et global de l'arbre vide sont égaux à zéro.

Question 14 Montrer que si T est globalement $\frac{1}{2}$-équilibré en taille, alors $D(T) = 0$.

Question 15 On suppose que T est un arbre obtenu après l'insertion d'un élément x dans un arbre bouc-émissaire, et qu'avant son rééquilibrage l'arbre possède un nœud bouc-émissaire s.

Montrer que $|T(s)| < 3D(s)$, où $T(s)$ désigne le sous-arbre enraciné en s, dans lequel on a rajouté l'élément x, avant d'en faire le rééquilibrage.

On considère une suite d'opérations d'insertions $I_1, I_2, \ldots, I_k$ dans un arbre initialement vide, et on note $T_0, T_1, \ldots, T_k$ les arbres ainsi obtenus (avec T_0 l'arbre vide initial). Pour une opération I_j, on note :

- $C(I_j)$ le nombre de nœuds parcourus lors de l'opération d'insertion, y compris les nœuds parcourus par des appels à `size` ou à `infix` (chaque nœud n'est compté qu'une seule fois, même si on l'atteint plusieurs fois par des appels différents) ;
- $A(I_j) = C(I_j) + 3(D(T_j) - D(T_{j-1}))$.

Question 16 Montrer que $\sum_{j=1}^{k} C(I_j) \leqslant \sum_{j=1}^{k} A(I_j)$. En déduire que l'opération d'insertion dans un arbre bouc-émissaire de taille n a une complexité temporelle amortie en $\mathcal{O}(\log n)$.

3 Relation de séparabilité par rapport à un langage

Il a été fixé, jusqu'à la fin de ce sujet, un certain langage régulier $L \subseteq \Sigma^*$, que nous ne connaissons pas mais que nous souhaitons apprendre. Autrement dit, notre objectif est de produire en temps fini et efficacement un automate qui reconnait le langage L. Nous accédons au langage L par l'intermédiaire d'un oracle qui répond à deux types de requêtes :

- des *requêtes d'appartenance*, prenant la forme de la question « le mot $w \in \Sigma^*$ appartient-il au langage fixé L ? »
- et des *requêtes d'équivalence*, prenant la forme de la question : « l'automate A reconnait-il le langage fixé L ? ».

En cas de réponse négative à une requête d'équivalence, l'oracle nous pourvoit d'un contre-exemple $c \in \Sigma^*$ pour lequel l'automate A se trompe. Autrement dit, soit le mot c appartient au langage à apprendre L mais n'est pas reconnu par l'automate A, soit le mot c n'appartient pas au langage à apprendre L mais est reconnu par l'automate A.

Remarque OCaml : L'oracle est empaqueté dans un module `Oracle` ainsi signé :

```
type answer = Yes | Counterexample of word

val mem : word -> bool
val equiv : automaton -> answer
```

et s'utilise avec la syntaxe `Oracle.mem w` pour une requête d'appartenance ou `Oracle.equiv a` pour une requête d'équivalence.

Définition : Nous notons $\mathbb{1}_\Lambda : \Sigma^* \to \{\mathsf{faux}, \mathsf{vrai}\}$ la *fonction indicatrice* de tout langage $\Lambda \subseteq \Sigma^*$. Pour tout mot $u \in \Sigma^*$, $\mathbb{1}_\Lambda(u)$ prend la valeur vrai si et seulement si le mot u appartient au langage Λ.

Définition : Deux mots quelconques $u_1 \in \Sigma^*$ et $u_2 \in \Sigma^*$ sont dits *séparés par le mot $\overline{v}$ par rapport à un langage* $\Lambda \subseteq \Sigma^*$ si l'on a

$$\mathbb{1}_\Lambda(u_1\overline{v}) \neq \mathbb{1}_\Lambda(u_2\overline{v})$$

autrement dit, si la concaténation $u_1\overline{v}$ appartient au langage Λ tandis que la concaténation $u_2\overline{v}$ n'appartient pas au langage Λ ou si la concaténation $u_1\overline{v}$ n'appartient pas au langage Λ tandis que la concaténation $u_2\overline{v}$ appartient au langage Λ. Nous disons alors que le mot $\overline{v}$ est un *discriminant*.

Remarque OCaml : Typographiquement, nous convenons de surligner les mots servant de discriminant dans ce sujet et marquons le rôle de discriminant en adoptant l'alias de type

```
type disc = word
```

Question 17 Écrire une fonction OCaml `separated_by (u1:word) (u2:word) (v:disc) : bool` qui teste si les deux mots u_1 et u_2 sont séparés par rapport au langage à apprendre L par le mot discriminant $\overline{v}$.

Définition : Deux mots quelconques $u_1 \in \Sigma^*$ et $u_2 \in \Sigma^*$ sont dits *séparables par rapport à un langage* $\Lambda \subseteq \Sigma^*$, ou simplement *séparables*, s'il existe un discriminant v qui les sépare. Ils sont dits *inséparables* sinon. Nous notons $u_1 \equiv_\Lambda u_2$ la relation d'inséparabilité par rapport au langage Λ.

Question 18 Montrer que la relation d'inséparabilité par rapport à un langage quelconque est une relation d'équivalence sur l'ensemble des mots Σ^*.

Question 19 Soient $A = (Q, q_0, \delta, \mathbb{1}_F)$ un automate quelconque, $L_A \subseteq \Sigma^*$ le langage reconnu par l'automate A et $(u_1, u_2) \in (\Sigma^*)^2$ deux mots tels que $\delta^*(q_0, u_1) = \delta^*(q_0, u_2)$. Montrer que les mots u_1 et u_2 sont inséparables par rapport au langage L_A.

4 Arbre discriminant

Définition : Un *arbre discriminant* est un arbre de décision binaire, dont les sommets internes sont étiquetés par des discriminants et dont les feuilles sont étiquetées par des états distincts deux à deux.

```
type disctree =
    | Node of disctree * disc * disctree
    | Leaf of state
```

La figure 2 présente un exemple d'arbre discriminant $\mathcal{T}_0$, dans lequel les trois sommets internes sont figurés par des cercles et les quatre feuilles par des rectangles.

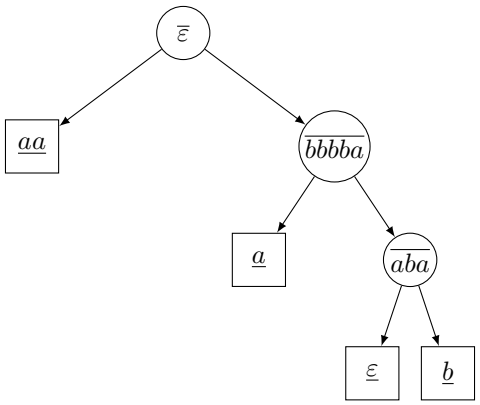
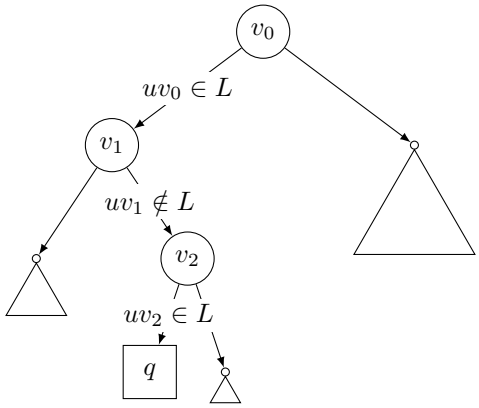


FIGURE 2 – Un exemple d'arbre discriminant : l'arbre $\mathcal{T}_0$

Définition : *Cribler un mot* $u \in \Sigma^*$ *à travers un arbre discriminant* T *pour un langage* L consiste à construire l'unique chemin $s_0 - s_1 - \cdots - s_k$ dans l'arbre T où :

– le sommet s_0 est la racine de T,
– pour tout indice i compris entre 0 et $k-1$, le sommet s_i est un sommet interne et, en notant v_i l'étiquette du sommet interne s_i, si le mot concaténé uv_i appartient au langage à apprendre L, alors le sommet s_{i+1} est le fils gauche de s_i, sinon, le sommet s_{i+1} est le fils droit de s_i,
– le sommet s_k est une feuille de l'arbre T.

et renvoyer l'état q porté par la feuille s_k. L'état q s'appelle le *criblat du mot* u et est noté $\lfloor u \rfloor$.



Par exemple, en supposant que le langage à apprendre L est dénoté par l'expression régulière ab^*a, le criblat du mot *abbba* par l'arbre discriminant $\mathcal{T}_0$ (figure 2) est l'état $\lfloor abbba \rfloor = \underline{aa}$, qui est l'extrémité du chemin $\overline{\varepsilon} - \underline{aa}$ obtenu en observant que *abbba* appartient à L. En revanche, le criblat du mot *abb* est l'état $\lfloor abb \rfloor = \underline{a}$, qui est l'extrémité du chemin $\overline{\varepsilon} - \overline{bbbba} - \underline{a}$ obtenu en observant que *abb* appartient à L mais *abbbbbba* n'appartient pas à L.

Question 20 Donner le criblat du mot *ba* pour l'exemple précédent.

Question 21 Écrire une fonction OCaml `sift (t:disctree) (u:word) : state` dont la valeur de retour est le criblat $\lfloor u \rfloor$ du mot u à travers l'arbre discriminant T pour le langage L.

Question 22 Déterminer la complexité en temps dans le pire des cas de la fonction `sift t u` en fonction de tout ou partie des grandeurs suivantes : la longueur $|u|$ du mot u, le nombre n de sommets dans l'arbre T, la hauteur h de l'arbre T, le maximum $M = \max_{v \in T} |v|$ des longueurs des mots discriminants de l'arbre T, le coût $\mu(|w|)$ d'un appel à `Oracle.mem w`, le coût $\nu(|\mathcal{A}|)$ d'un appel `Oracle.equiv a`.

Question 23 Démontrer, en exhibant un discriminant, que deux mots de criblats distincts sont toujours séparables.

Définition : Nous disons qu'un arbre discriminant est *accessible* si, pour tout état $\underline{w} \in \Sigma^*$, apparaissant dans la liste des étiquettes des feuilles, le criblat $\lfloor w \rfloor$ du mot w est l'état $\underline{w}$.

Question 24 Démontrer qu'il existe une constante dépendant uniquement du langage à apprendre L, qui majore le nombre de feuilles d'un arbre discriminant accessible.

Définition : Nous disons qu'un arbre discriminant est *démêlant* s'il comporte au moins un sommet interne et si l'étiquette de la racine est le discriminant mot vide $\overline{\varepsilon}$.

Question 25 Démontrer que si deux mots u_1 et $u_2 \in \Sigma^*$ ont le même criblat par rapport à un arbre discriminant démêlant, alors on a l'égalité $\mathbb{1}_L(u_1) = \mathbb{1}_L(u_2)$.

5 Automate tiré d'un arbre discriminant

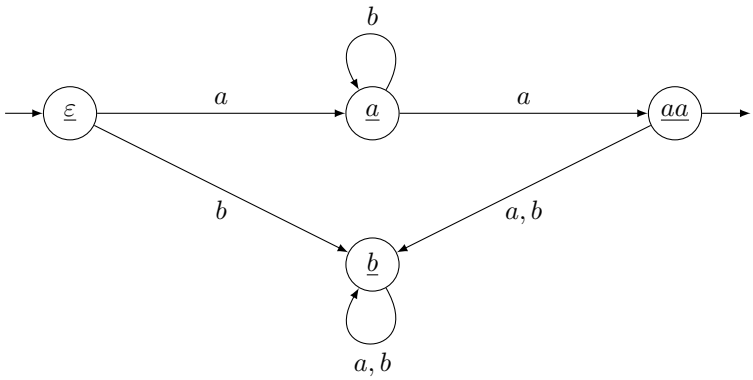
Définitions : Nous disons qu'un arbre discriminant T est un *crible* s'il est accessible, démêlant et si le mot vide $\underline{\varepsilon}$ est un état qui apparait dans la liste des étiquettes des feuilles.

L'*automate associé à un crible* T *pour un langage* L est l'automate A ainsi défini :
– l'ensemble des états de A est l'ensemble des étiquettes des feuilles de T ;
– l'état initial de A est l'état $\underline{\varepsilon}$;
– la fonction de transition associe l'état $\underline{w}$ et la lettre x à l'état $\lfloor wx \rfloor$ obtenu en criblant le mot wx à travers

l'arbre discriminant T pour le langage L ;

– l'ensemble des états finals est l'ensemble des mots $\underline{w}$ où $\underline{w}$ est une étiquette d'une feuille du sous-arbre gauche de T.

Par exemple, l'automate associé au crible $\mathcal{T}_0$ de la figure 2 pour le langage dénoté par ab^*a est l'automate ainsi figuré



Question 26 Écrire une fonction OCaml `automaton_of_disctree (t:disctree) : automaton` dont la valeur de retour est l'automate associé au crible T.
On recopiera et on complètera le code suivant :

```
let automaton_of_disctree (t:disctree) : automaton = {
    initial = .... ;
    transitions = .... ;
    finals = ....
}
```

Question 27 En supposant que le langage à apprendre L n'est ni le langage vide, ni le langage plein Σ^*, décrire une construction, à l'aide des fonctions `Oracle.mem` et `Oracle.equiv`, d'un premier crible formé d'un seul sommet interne et de deux feuilles.

Dans les questions 28, 29 et 30, nous notons $A = (Q, \underline{\varepsilon}, \delta, \mathbb{1}_F)$ l'automate associé à un certain crible T déjà construit. Nous supposons que `Oracle.equiv a` renvoie `Oracle.Counterexample c`, où c est le mot non vide $c = c_0c_1 \ldots c_{\gamma-1} \in \Sigma^*$. Nous nous proposons de construire un nouveau crible T' ayant un état de plus que le crible T.

Pour tout indice i compris entre 0 et γ, nous appelons $\pi_i = c_0c_1 \ldots c_{i-1}$ le préfixe de c de longueur i et $\sigma_i = c_{i+1}c_{i+2} \ldots c_{\gamma-1}$ le suffixe de longueur $\gamma - i - 1$ de sorte que le mot c admette la factorisation $c = \pi_i c_i \sigma_i$. Par convention, on pose $c_\gamma \sigma_\gamma = \varepsilon$.

Nous notons les états

$$\underline{p_i} = \lfloor \pi_i \rfloor \in \Sigma^* \qquad \text{et} \qquad \underline{\hat{p}_i} = \delta^*(\underline{\varepsilon}, \pi_i) \in \Sigma^*$$

et posons encore

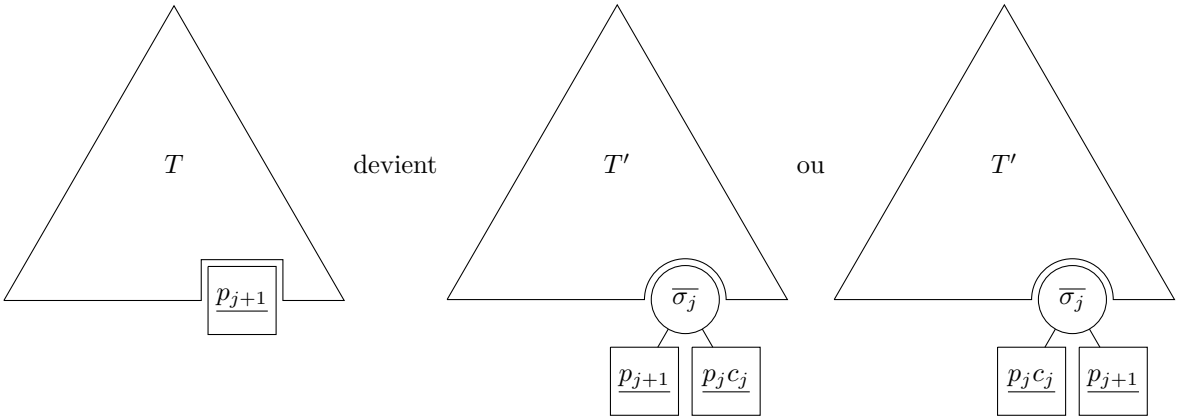
$$\tau_i = p_i c_i \sigma_i \in \Sigma^* \qquad \text{et} \qquad \hat{\tau}_i = \hat{p}_i c_i \sigma_i \in \Sigma^*$$

Question 28 Démontrer que l'on a $\mathbb{1}_L(\tau_\gamma) \neq \mathbb{1}_L(\hat{\tau}_\gamma)$.

On admet qu'il existe un indice i compris entre 0 et $\gamma - 1$ tel que les états $\underline{p_i}$ et $\underline{\hat{p}_i}$ sont égaux, les états $\underline{p_{i+1}}$ et $\underline{\hat{p}_{i+1}}$ sont distincts et les mots p_ic_i et p_{i+1} sont séparés par le discriminant $\overline{\sigma_i}$.

Question 29 Écrire une fonction `split (t:disctree) (c:word) : state * state * disc` dont la valeur de retour est le triplet $(\underline{p_j c_j}, \underline{p_{j+1}}, \overline{\sigma_j})$ où l'entier j est le plus petit des indices i compris entre 0 et $\gamma - 1$ obtenu par la question 28.

On admet de la question 28 que l'arbre discriminant T' obtenu en substituant un arbre constitué d'un sommet interne d'étiquette $\overline{\sigma_j}$, d'une feuille d'étiquette l'ancien état $\underline{p_{j+1}}$ et d'une feuille d'étiquette un nouvel état $\underline{p_j c_j}$ à la place de la feuille d'étiquette $\underline{p_{j+1}}$ dans l'arbre discriminant T jouit encore des propriétés de crible. Schématiquement,



Question 30 Écrire une fonction OCaml `substitute (t:disctree) ((pp,p,s):state * state * disc) : disctree` ainsi spécifiée :
Précondition : Le triplet (pp, p, s) est la valeur de retour de la fonction `split` écrite à la question 29.
Valeur de retour : Crible T' décrit ci-dessus.

Question 31 Concevoir et décrire en français un algorithme complet qui apprend un langage régulier à base de requêtes d'appartenance et de requêtes d'équivalence en renvoyant un automate. Justifier sa terminaison à l'aide d'un variant de boucle.

Question 32 Démontrer que l'automate construit à la question 31 possède un nombre d'états minimal parmi l'ensemble des automates qui reconnaissent le langage à apprendre.

A. Annexe : aide à la programmation en OCaml

Nous reproduisons des extraits de la documentation OCaml.

Module `List` : Le module `List` offre les fonctions suivantes :

- `append : 'a list -> 'a list -> 'a list`
  `append l0 l1` appends `l1` to `l0`. Same function as the infix operator `@`.
- `rev : 'a list -> 'a list`
  List reversal.
- `iter : ('a -> unit) -> 'a list -> unit`
  `iter f [a1; ...; an]` applies function `f` in turn to `[a1; ...; an]`. It is equivalent to `f a1; f a2; ...; f an`.
- `map : ('a -> 'b) -> 'a list -> 'b list`
  `map f [a1; ...; an]` applies function `f` to `a1`,...,`an`, and builds the list `[f a1; ...; f an]` with the results returned by `f`.
- `for_all : ('a -> bool) -> 'a list -> bool`
  `for_all f [a1; ...; an]` checks if all elements of the list satisfy the predicate `f`. That is, it returns `(f a1) && (f a2) && ... && (f an)` for a non-empty list and `true` if the list is empty.
- `exists : ('a -> bool) -> 'a list -> bool`
  `exists f [a1; ...; an]` checks if at least one element of the list satisfies the predicate `f`. That is, it returns `(f a1) || (f a2) || ... || (f an)` for a non-empty list and `false` if the list is empty.
- `mem : 'a -> 'a list -> bool`
  `mem a set` is true if and only if `a` is equal to an element of `set`.
- `filter : ('a -> bool) -> 'a list -> 'a list`
  `filter f l` returns all the elements of the list `l` that satisfy the predicate `f`. The order of the elements in the input list is preserved.

Module `Array` : Le module `Array` offre les fonctions suivantes :

- `length : 'a array -> int`
  Return the length (number of elements) of the given array.
- `make : int -> 'a -> 'a array`
  `make n x` returns a fresh array of length n, initialized with x.

D'après `https://ocaml.org/manual/5.1/api/List.html`

⋆ ⋆ ⋆